



Week 03 — Statistical Inference

Turning a point estimate into a confidence interval
— analytically, by simulation, and by bootstrap

AUTHORS

Johann-Friedrich Salzmänn  

Joshua Elias Schmidt Rodrigues  

DATE

2026-09-22

Topics:

Package management	2
Where we left off	2
Statistical inference: vocabulary	2
Data: pre-election polls (2025)	2
Point estimate and standard error	4
Confidence intervals for a mean	5
Confidence interval for a ratio of proportions	11
Data: German election results (2025)	12
Regional variation: East vs. West Germany	13
Difference in means: the East-West AfD gap	14
Confidence intervals for a difference in means	15
Concluding remarks	21
Try it: a 99% CI for the AfD's polled mean share	21
Try it: bootstrap CI for a share of strong polls	22
Try it: repeat the difference-in-means for the Greens	22

Package management

Same as every session: bootstrap pacman, then load the tidyverse.

```
if (!require(pacman)) install.packages("pacman")
```

MESSAGE

Loading required package: pacman

```
p_load(tidyverse)
set.seed(303)
```

Where we left off

Week 1 covered descriptive statistics (mean, variance, standard deviation) on real polling data. Week 2 introduced probability distributions and, with the Central Limit Theorem, the key fact that a sample mean's own variability — its *standard error* — shrinks as the sample size grows. Both weeks stopped at a single number: one mean, one proportion, one point. This week asks the natural next question: how much do we trust that single number, and how do we express that trust as a *range* of plausible values instead of a false sense of precision?

Statistical inference: vocabulary

- An **estimand** is the actual quantity we want to learn about — defined by the research question itself, before any data or model comes in. Here: the AfD's true level of support at a given moment.
- A **parameter** is that same estimand once expressed as a fixed, unknown feature of a population or probability model — e.g. the true share of the electorate that supports a party at a given moment, which no poll ever observes directly.
- An **estimator** is a function applied to a sample that produces a guess at that parameter — `mean()` is the estimator for a population mean.
- An **estimate** is the actual number an estimator returns for one particular sample.

Because we only ever observe one sample, our estimate carries uncertainty. A **confidence interval (CI)** turns that single estimate into a range: an interval that will contain the true parameter in some fixed proportion of repeated samples.

Data: pre-election polls (2025)

Same ZEIT ONLINE polling data as Week 1 — one row per poll. Our estimand is the AfD's true level of support in the three weeks immediately before the 2025-02-23 federal election — what did the polls say about it, and how confident can we be in that read on the electorate's true mood?

```
polls <- read_csv("data/polls.csv")
```

MESSAGE

```
Rows: 2077 Columns: 12
-- Column specification -----
Delimiter: ","
chr  (3): Poll_ID, Pollster, Period
dbl  (8): n, CDUCSU, SPD, Gruene, FDP, Linke, AfD, BSW
date (1): Date

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
polls
```

OUTPUT

```
# A tibble: 2,077 x 12
  Poll_ID Pollster Date      n Period CDUCSU SPD Gruene FDP Linke
  <chr>    <chr>   <date>   <dbl> <chr>   <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa    2017-01-04 2001 "27.1~ 37 21 10 6 9
2 BT01170105P Infrates~ 2017-01-05 1505 "02.0~ 37 20 9 5 9
3 BT03170107L Verian    2017-01-07 834 "03.0~ 38 22 10 5 9
4 BT02170111P Forsa    2017-01-11 2501 "02.0~ 37 20 10 6 9
5 BT03170114M Verian    2017-01-14 1878 "06.0~ 37 21 10 6 10
6 BT02170118Q Forsa    2017-01-18 2503 "09.0~ 38 21 9 6 9
7 BT03170121N Verian    2017-01-21 2805 "12.0~ 36 21 9 6 11
8 BT02170125R Forsa    2017-01-25 2504 "16.0~ 37 21 10 6 9
9 BT01170127Q Infrates~ 2017-01-27 1029 "24.0~ 35 23 9 6 8
10 BT031701280 Verian    2017-01-28 1996 "19.0~ 37 23 10 6 10
# i 2,067 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

```
polls_long <- polls |>
  pivot_longer(CDU:BSW, names_to = "party", values_to = "share")

afd_polls <- polls_long |>
  filter(party == "AfD", between(Date, as.Date("2025-02-02"), as.Date("2025-02-23")))
afd_polls
```

> OUTPUT

```
# A tibble: 25 x 7
  Poll_ID Pollster Date n Period party share
  <chr>    <chr>   <date> <dbl> <chr>   <chr> <dbl>
1 BT05250203G INSA 2025-02-03 2004 "31.01.\u20~ AfD 22
2 BT022502040 Forsa 2025-02-04 2503 "28.01.\u20~ AfD 20
3 BT13250205D YouGov 2025-02-05 2181 "31.01.\u20~ AfD 22
4 BT01250206R Infratest dimap 2025-02-06 1302 "03.02.\u20~ AfD 21
5 BT06250207G Forschungsgruppe Wahlen 2025-02-07 1341 "04.02.\u20~ AfD 20
6 BT04250207Y GMS 2025-02-07 1011 "04.02.\u20~ AfD 21
7 BT05250208H INSA 2025-02-08 1204 "03.02.\u20~ AfD 21
8 BT05250210I INSA 2025-02-10 2006 "07.02.\u20~ AfD 22
9 BT02250211P Forsa 2025-02-11 2502 "04.02.\u20~ AfD 20
10 BT13250212E YouGov 2025-02-12 2083 "07.02.\u20~ AfD 21
# i 15 more rows
```

Point estimate and standard error

Week 2's Central Limit Theorem gave us the sampling distribution of a mean, $\mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right)$ — its standard deviation, the *standard error*, is estimated from a single sample as:

$$SE(\hat{\theta}) = \frac{\hat{\sigma}}{\sqrt{N}}$$

```
afd_stats <- afd_polls |>
  summarize(
    mean_share = mean(share),
    sd_share   = sd(share),
    n          = n()
  )
afd_stats
```

> OUTPUT

```
# A tibble: 1 x 3
  mean_share sd_share n
  <dbl>    <dbl> <int>
1    20.7    0.737   25
```

Pull the three pieces we need out of that summary:

```
mean_share <- afd_stats |> pull(mean_share)
sd_share   <- afd_stats |> pull(sd_share)
n_polls    <- afd_stats |> pull(n)

se_share <- sd_share / sqrt(n_polls)
se_share
```

> OUTPUT

```
[1] 0.1474223
```

Confidence intervals for a mean

1. **Analytically** — quantiles of the Normal approximation.
2. **By simulation** — draw from the assumed sampling distribution directly.
3. **By bootstrap** — resample the polls themselves, with replacement.

We compare all three once we have them.

1. Analytical

$$\hat{\theta} \pm 1.96 \text{SE}(\hat{\theta})$$

with 1.96 the 2.5%/97.5% quantiles of the standard Normal.

```
ci_analytical <- mean_share + qnorm(c(0.025, 0.975)) * se_share
ci_analytical
```

> OUTPUT

```
[1] 20.43106 21.00894
```

2. Interpreting a confidence interval

A 95% CI does **not** mean “95% probability the true mean is in this particular interval” — the true mean either is or isn’t in it, no probability about it. It means: if we repeated the whole sample-and-build-a-CI process many times, about 95% of the resulting intervals would contain the true parameter. We can only show this by pretending to know the truth — so, purely for this demonstration, we invent a population with a known true mean, using our own data’s poll-to-poll variability as the population’s spread:

```
true_mean <- 21
demo_pop <- rnorm(10000, true_mean, sd_share)
```

`expand_grid()` lays out every (trial, draw) combination at once — `group_by(trial) + summarize()` then computes each trial’s own sample mean and CI in one vectorized pass, no loop needed. Each trial resamples as many polls as our real window actually had:

```

n_trials <- 100

ci_trials <- expand_grid(
  trial = 1:n_trials,
  draw = 1:n_polls
) |>
  group_by(trial) |>
  mutate(value = sample(demo_pop, n(), replace = TRUE)) |>
  summarize(
    est      = mean(value),
    se       = sd(value) / sqrt(n()),
    lo       = est + qnorm(0.025) * se,
    hi       = est + qnorm(0.975) * se,
    .groups = "drop"
  ) |>
  mutate(missed = lo > true_mean | hi < true_mean)

ci_trials |> count(missed)

```

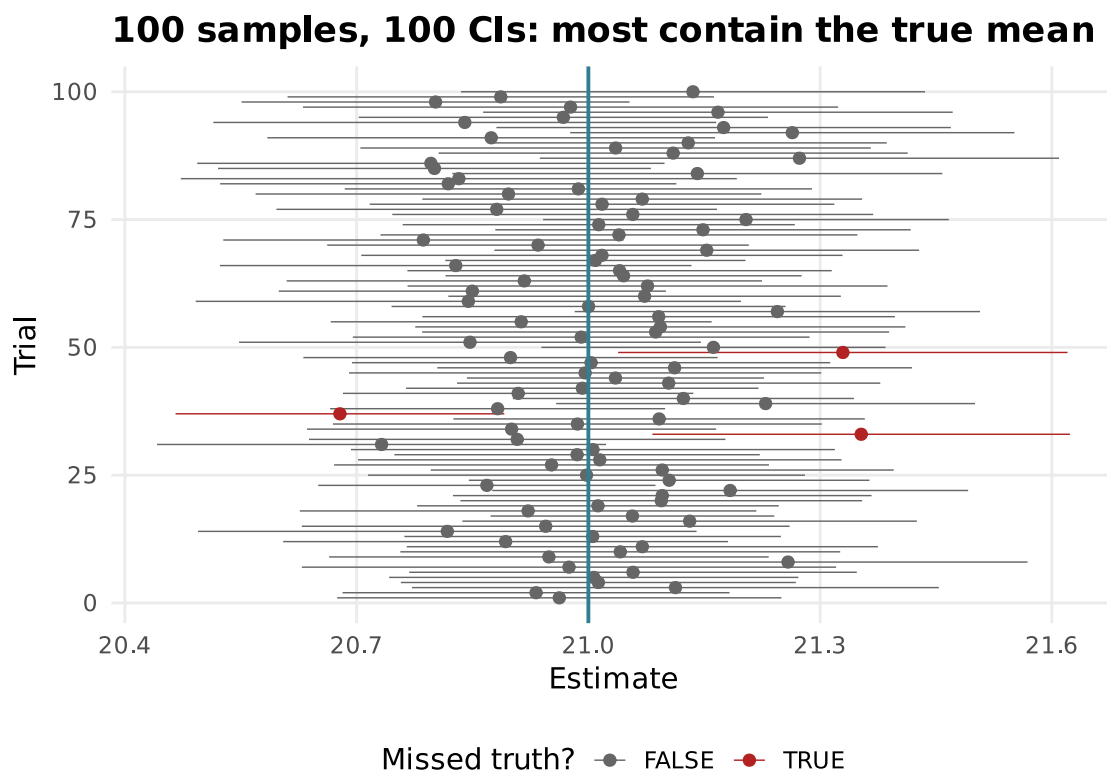
> OUTPUT

```

# A tibble: 2 x 2
  missed      n
  <lgl>   <int>
1 FALSE    97
2 TRUE      3

```

```
ci_trials |>
  ggplot(aes(x = est, y = trial, xmin = lo, xmax = hi, color = missed)) +
  geom_pointrange(linewidth = 0.3, size = 0.3) +
  geom_vline(xintercept = true_mean, color = course_primary, linewidth = 0.8) +
  scale_color_manual(values = c("FALSE" = "grey40", "TRUE" = "firebrick")) +
  labs(
    title = "100 samples, 100 CIs: most contain the true mean",
    x = "Estimate",
    y = "Trial",
    color = "Missed truth?"
  ) +
  theme_intror()
```



3. Simulation

The CLT tells us the sampling distribution of the mean is approximately $\mathcal{N}(\hat{\theta}, \text{SE}(\hat{\theta})^2)$ — draw from that assumed distribution directly, then take empirical quantiles:

```
nsim      <- 1000
sim_means <- rnorm(nsim, mean_share, se_share)

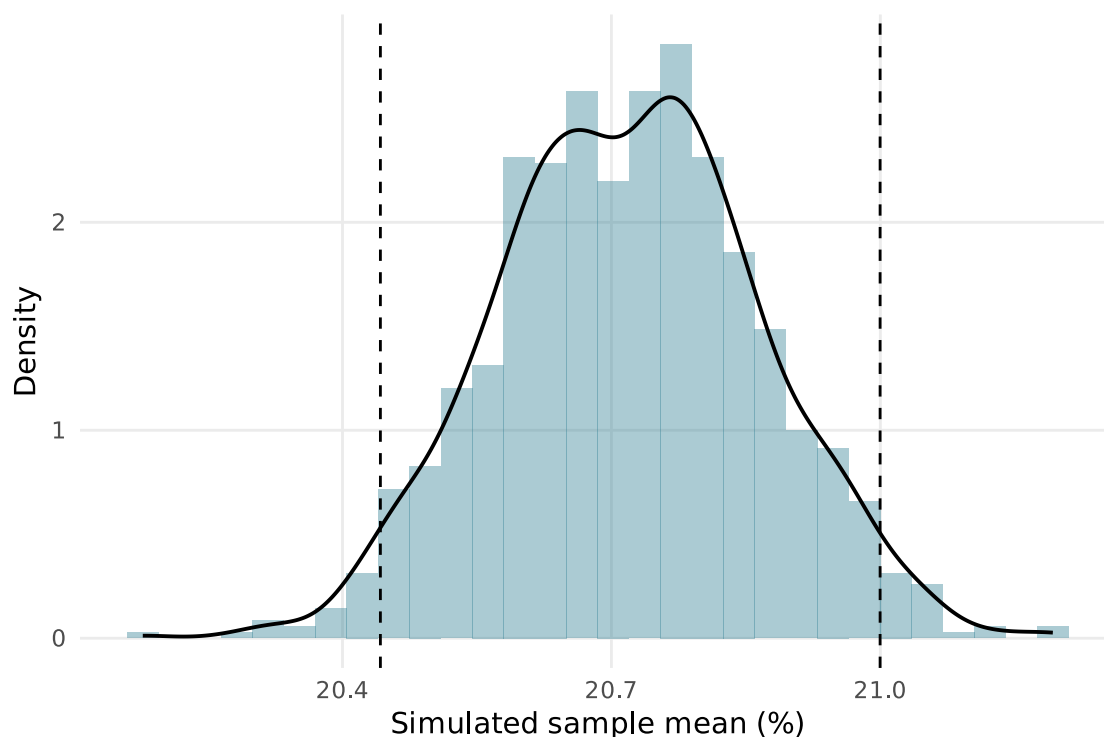
ci_simulated <- quantile(sim_means, c(0.025, 0.975))
ci_simulated
```

> OUTPUT

```
2.5%    97.5%
20.44226 20.99974
```

```
tibble(sim_means) |>
  ggplot(aes(x = sim_means)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30, fill = course_primary, alpha
= 0.4, color = NA) +
  geom_density(linewidth = 0.8) +
  geom_vline(xintercept = ci_simulated, linetype = "dashed") +
  labs(
    title = "Simulated sampling distribution of the mean",
    x      = "Simulated sample mean (%)",
    y      = "Density"
  ) +
  theme_intror()
```

Simulated sampling distribution of the mean



4. Bootstrap

Resample the polls themselves, with replacement, many times — same vectorized “one big tibble, `group_by(trial)`” pattern as the coverage demo above, just resampling real observed shares instead of a known population. `sample()` draws from a plain vector, so `afd_shares` is pulled out once first:

```
afd_shares <- afd_polls |> pull(share)
```

```

n_boot <- 1000

boot_means <- expand_grid(
  trial = 1:n_boot,
  row   = 1:n_polls
) |>
  group_by(trial) |>
  summarize(boot_mean = mean(sample(afd_shares, n(), replace = TRUE)), .groups = "drop")

boot_mean_values <- boot_means |> pull(boot_mean)
ci_bootstrap <- quantile(boot_mean_values, c(0.025, 0.975))
ci_bootstrap

```

> OUTPUT

```

2.5% 97.5%
20.44 21.00

```

Comparing the three CIs

```

ci_comparison <- tibble(
  method = c("Analytical", "Simulation", "Bootstrap"),
  lower   = c(ci_analytical[1], ci_simulated[1], ci_bootstrap[1]),
  upper   = c(ci_analytical[2], ci_simulated[2], ci_bootstrap[2])
)
ci_comparison

```

> OUTPUT

```

# A tibble: 3 x 3
  method lower upper
  <chr>   <dbl> <dbl>
1 Analytical 20.4 21.0
2 Simulation 20.4 21.0
3 Bootstrap 20.4 21

```

All three agree closely — the interval is narrow because $\sqrt{N}$ already does a lot of work even at $N = 25$ polls, exactly the same reason bigger surveys are more precise.

Putting all three intervals next to the polls' own empirical distribution makes the comparison concrete — along with the actual election result, which we haven't computed yet. For now it's simply hard-coded; the second half of this lab derives it properly from the real county-level results:

```

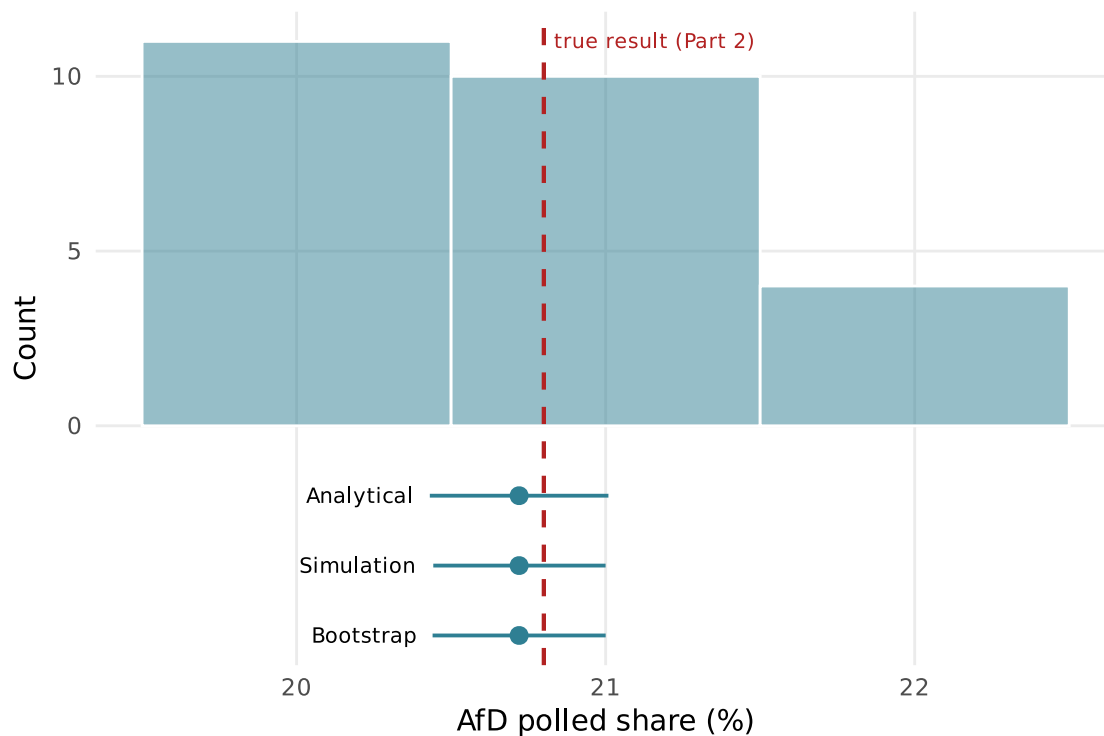
true_value <- 20.8 # the actual 2025 result – computed properly in Part 2

ci_comparison_plot <- ci_comparison |>
  mutate(
    mid = mean_share,
    y   = -2 * row_number()
  )

afd_polls |>
  ggplot(aes(x = share)) +
  geom_histogram(binwidth = 1, boundary = 0.5, fill = course_primary, alpha = 0.5, color = "white") +
  geom_vline(xintercept = true_value, color = "firebrick", linewidth = 0.9, linetype = "dashed") +
  annotate("text", x = true_value, y = 11, label = "true result (Part 2)", color = "firebrick", size = 3.2, hjust = -0.05) +
  geom_pointrange(
    data = ci_comparison_plot,
    aes(x = mid, y = y, xmin = lower, xmax = upper),
    inherit.aes = FALSE,
    color = course_primary,
    linewidth = 0.8
  ) +
  geom_text(
    data = ci_comparison_plot,
    aes(x = lower, y = y, label = method),
    inherit.aes = FALSE, hjust = 1.15, size = 3.3
  ) +
  scale_y_continuous(breaks = seq(0, 10, 5)) +
  labs(
    title = "The polls, three CIs, and the true result",
    x     = "AfD polled share (%)",
    y     = "Count"
  ) +
  theme_intror()

```

The polls, three CIs, and the true result



Confidence interval for a ratio of proportions

Not every quantity of interest has a clean standard-error formula. A *ratio* of two proportions is a case where the analytical derivation gets awkward fast, but simulation still works exactly the same way. This example is from the lecture: in a sample of 500 men and 500 women, 55% of men and 65% of women report voting for a left-wing party. We want a CI for the ratio of these two proportions, 0.55/0.65.

$$SE(\hat{p}) = \sqrt{\hat{p} \frac{1 - \hat{p}}{n}}$$

```
p_hat_men    <- 0.55
p_hat_women  <- 0.65
n_men        <- 500
n_women      <- 500

se_men    <- sqrt(p_hat_men * (1 - p_hat_men) / n_men)
se_women  <- sqrt(p_hat_women * (1 - p_hat_women) / n_women)
```

```
ratio_sim <- tibble(
  p_men   = rnorm(nsim, p_hat_men, se_men),
  p_women = rnorm(nsim, p_hat_women, se_women)
) |>
  mutate(ratio = p_men / p_women)

ratio_values <- ratio_sim |> pull(ratio)
ci_ratio <- quantile(ratio_values, c(0.025, 0.975))
ci_ratio
```

> OUTPUT

```
      2.5%      97.5%
0.7646757 0.9391692
```

Data: German election results (2025)

GERDA, the same harmonized county-level German federal election results Week 1's homework used. We load it, restrict to the 2025 election, and compute the true, vote-weighted national AfD share — the election is behind us now, so how did those final-stretch polls do against what actually happened?

```
gerda <- read_csv("data/federal_cty_harm.csv")
```

MESSAGE

```
Rows: 4000 Columns: 127
-- Column specification -----
Delimiter: ","
chr   (2): county_code, state
dbl  (124): election_year, eligible_voters, number_voters, valid_votes, inva...
date  (1): election_date

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
gerda_2025 <- gerda |>
  filter(election_year == 2025)
```

```
national_afd <- gerda_2025 |>
  summarize(afd_national_pct = 100 * sum(afd * total_votes) / sum(total_votes))
national_afd
```

> OUTPUT

```
# A tibble: 1 x 1
  afd_national_pct
      <dbl>
1          20.8
```

That figure lands right inside every one of the three CIs built above — aggregated properly, the final-stretch polls called it almost exactly. Not every election is this well-behaved, but it's a good sanity check on the whole machinery: a confidence interval is only useful if it actually tends to capture the truth.

Regional variation: East vs. West Germany

The national figure hides a lot. Is AfD support actually uniform across the country, or does it differ systematically between Eastern and Western Germany? Berlin (state code 11) is excluded — its historical East/West split makes a county-level label ambiguous rather than a clean binary.

```
west_codes <- c("01", "02", "03", "04", "05", "06", "07", "08", "09", "10")
east_codes <- c("12", "13", "14", "15", "16")

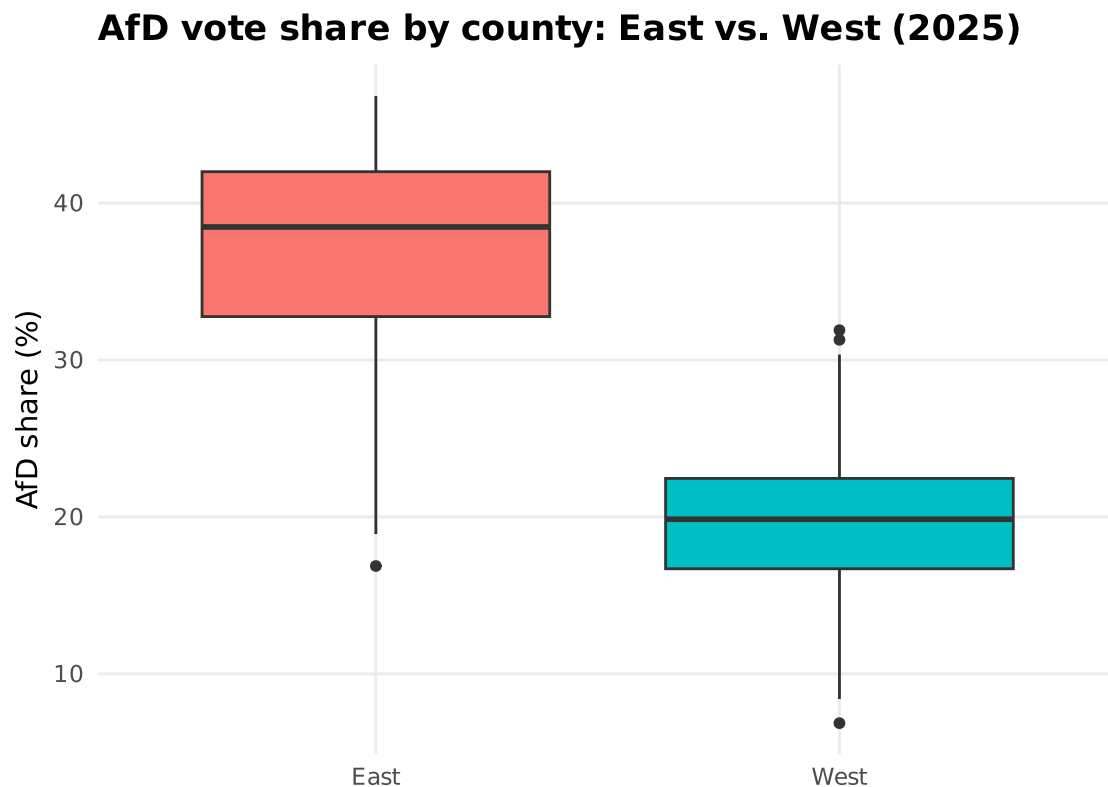
gerda_regions <- gerda_2025 |>
  filter(state %in% c(west_codes, east_codes)) |>
  mutate(
    region = case_when(
      state %in% west_codes ~ "West",
      state %in% east_codes ~ "East"
    ),
    afd_pct = afd * 100
  ) |>
  select(county_code, state, region, afd_pct)

gerda_regions |> count(region)
```

> OUTPUT

```
# A tibble: 2 x 2
  region      n
  <chr>  <int>
1 East      75
2 West     324
```

```
gerda_regions |>
  ggplot(aes(x = region, y = afd_pct, fill = region)) +
  geom_boxplot() +
  labs(
    title = "AfD vote share by county: East vs. West (2025)",
    x      = NULL,
    y      = "AfD share (%)"
  ) +
  theme_intror() +
  theme(legend.position = "none")
```



Difference in means: the East-West AfD gap

Our estimand this time is the true difference in AfD support between Eastern and Western Germany:

```
diff_stats <- gerda_regions |>
  group_by(region) |>
  summarize(
    mean_afd = mean(afd_pct),
    var_afd  = var(afd_pct),
    n        = n()
  )
diff_stats
```

> OUTPUT

```
# A tibble: 2 x 4
  region mean_afd var_afd    n
  <chr>   <dbl>   <dbl> <int>
1 East    37.1    49.0    75
2 West    19.5    19.4   324
```

```
mean_east <- diff_stats |>
  filter(region == "East") |>
  pull(mean_afd)
mean_west <- diff_stats |>
  filter(region == "West") |>
  pull(mean_afd)
var_east <- diff_stats |>
  filter(region == "East") |>
  pull(var_afd)
var_west <- diff_stats |>
  filter(region == "West") |>
  pull(var_afd)
n_east <- diff_stats |>
  filter(region == "East") |>
  pull(n)
n_west <- diff_stats |>
  filter(region == "West") |>
  pull(n)

diff_observed <- mean_east - mean_west
diff_observed
```

> OUTPUT

```
[1] 17.51957
```

Confidence intervals for a difference in means

1. **Analytically** — Normal approximation, combining both groups' own variance.
2. **By simulation** — draw each group's sampling distribution, subtract.
3. **By bootstrap** — resample each group with replacement, subtract.

4. **t.test()** — R's built-in shortcut for the analytical version, previewing the formal hypothesis test Week 6 covers in full.

1. Analytical

$$SE(\hat{\theta}_1 - \hat{\theta}_2) = \sqrt{\frac{\hat{\sigma}_1^2}{n_1} + \frac{\hat{\sigma}_2^2}{n_2}}$$

```
se_diff <- sqrt(var_east / n_east + var_west / n_west)
ci_diff_analytical <- diff_observed + qnorm(c(0.025, 0.975)) * se_diff
ci_diff_analytical
```

> OUTPUT

```
[1] 15.86446 19.17468
```

2. Simulation

```
se_east <- sqrt(var_east / n_east)
se_west <- sqrt(var_west / n_west)

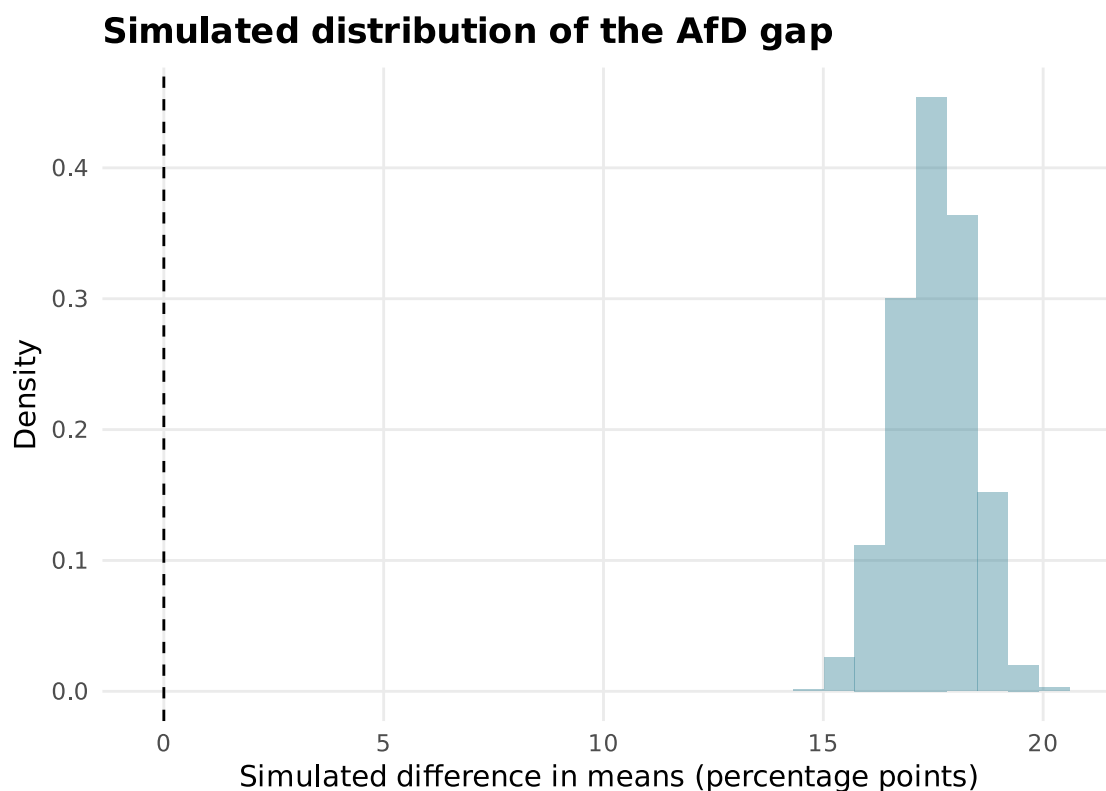
diff_sim <- tibble(
  sim_east = rnorm(nsim, mean_east, se_east),
  sim_west = rnorm(nsim, mean_west, se_west)
) |>
  mutate(diff = sim_east - sim_west)

diff_values <- diff_sim |> pull(diff)
ci_diff_simulated <- quantile(diff_values, c(0.025, 0.975))
ci_diff_simulated
```

> OUTPUT

```
      2.5%    97.5%
15.93585 19.06114
```

```
diff_sim |>
  ggplot(aes(x = diff)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30, fill = course_primary, alpha
= 0.4, color = NA) +
  geom_vline(xintercept = 0, linetype = "dashed") +
  labs(
    title = "Simulated distribution of the AfD gap",
    x      = "Simulated difference in means (percentage points)",
    y      = "Density"
  ) +
  theme_intror()
```



Zero sits nowhere near this distribution — a first, informal hint that the gap is not just noise, well before Week 6 turns that hint into a formal test.

3. Bootstrap

Resampling here works on two bare vectors, one per region — pulled out specifically because `t.test()` and `sample()` below need vectors, not tibbles:

```
afd_east <- gerda_regions |>
  filter(region == "East") |>
  pull(afd_pct)
afd_west <- gerda_regions |>
  filter(region == "West") |>
  pull(afd_pct)
```

```

n_boot_diff <- 1000

boot_east <- expand_grid(trial = 1:n_boot_diff, row = 1:n_east) |>
  group_by(trial) |>
  summarize(boot_mean = mean(sample(afd_east, n(), replace = TRUE)), .groups = "drop")

boot_west <- expand_grid(trial = 1:n_boot_diff, row = 1:n_west) |>
  group_by(trial) |>
  summarize(boot_mean = mean(sample(afd_west, n(), replace = TRUE)), .groups = "drop")

boot_east_values <- boot_east |> pull(boot_mean)
boot_west_values <- boot_west |> pull(boot_mean)
diff_boot_values <- boot_east_values - boot_west_values

ci_diff_bootstrap <- quantile(diff_boot_values, c(0.025, 0.975))
ci_diff_bootstrap

```

> OUTPUT

```

      2.5%      97.5%
15.85954 19.02509

```

4. t.test()

```

t_result <- t.test(afd_east, afd_west)
t_result

```

> OUTPUT

```

Welch Two Sample t-test

data:  afd_east and afd_west
t = 20.747, df = 88.034, p-value < 0.00000000000000022
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 15.84139 19.19774
sample estimates:
mean of x mean of y
 37.06596 19.54639

```

```

ci_diff_t <- t_result |>
  pluck("conf.int") |>
  as.numeric()
ci_diff_t

```

> OUTPUT

```
[1] 15.84139 19.19774
```

Comparing the four CIs

```
diff_comparison <- tibble(  
  method = c("Analytical", "Simulation", "Bootstrap", "t.test()"),  
  lower   = c(ci_diff_analytical[1], ci_diff_simulated[1], ci_diff_bootstrap[1],  
ci_diff_t[1]),  
  upper   = c(ci_diff_analytical[2], ci_diff_simulated[2], ci_diff_bootstrap[2],  
ci_diff_t[2])  
)  
diff_comparison
```

>_ OUTPUT

```
# A tibble: 4 x 3  
  method      lower upper  
  <chr>      <dbl> <dbl>  
1 Analytical  15.9  19.2  
2 Simulation  15.9  19.1  
3 Bootstrap   15.9  19.0  
4 t.test()    15.8  19.2
```

All four intervals land in the same place, comfortably clear of zero.

Same comparison as before, but there's no separate “true” value to reveal this time — GERDA's 2025 county results already *are* the full election outcome, not a sample of it, so the observed gap itself is as true as it gets. The backdrop here is the bootstrap distribution, the most directly “from the data itself” of the four methods:

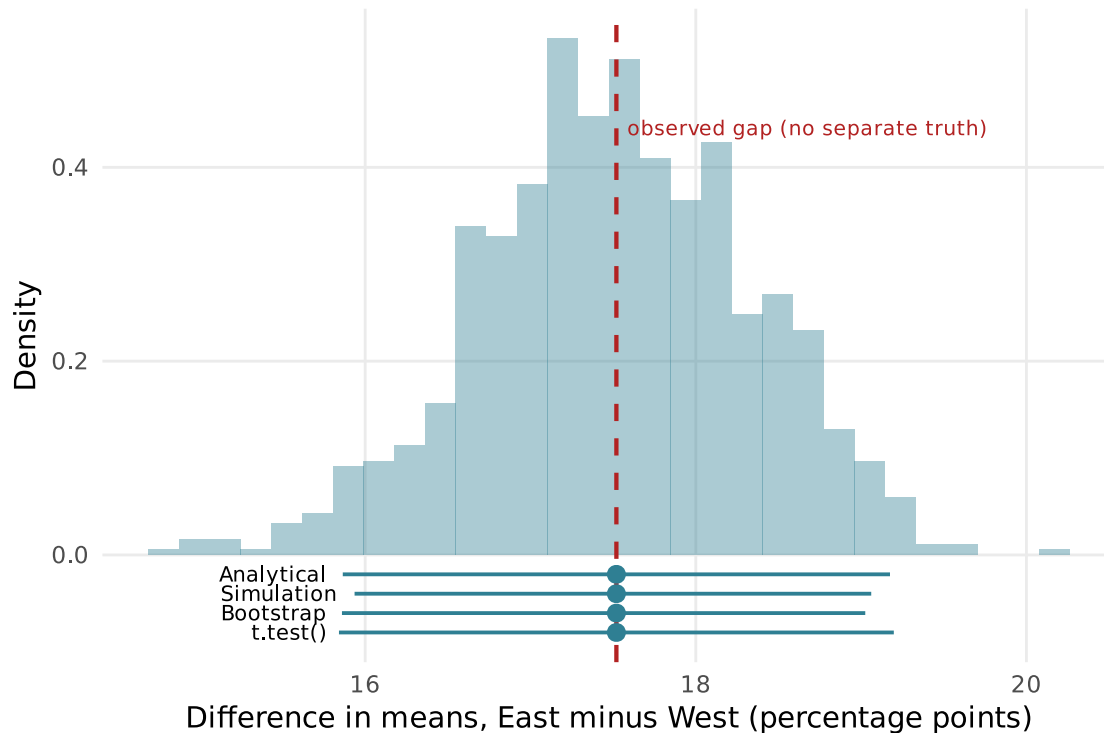
```

diff_comparison_plot <- diff_comparison |>
  mutate(
    mid = diff_observed,
    y   = -0.02 * row_number()
  )

tibble(diff = diff_boot_values) |>
  ggplot(aes(x = diff)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30, fill = course_primary, alpha
= 0.4, color = NA) +
  geom_vline(xintercept = diff_observed, color = "firebrick", linewidth = 0.9, linetype
= "dashed") +
  annotate("text", x = diff_observed, y = 0.44, label = "observed gap (no separate
truth)", color = "firebrick", size = 3.2, hjust = -0.03) +
  geom_pointrange(
    data = diff_comparison_plot,
    aes(x = mid, y = y, xmin = lower, xmax = upper),
    inherit.aes = FALSE,
    color = course_primary,
    linewidth = 0.8
  ) +
  geom_text(
    data = diff_comparison_plot,
    aes(x = lower, y = y, label = method),
    inherit.aes = FALSE, hjust = 1.15, size = 3.3
  ) +
  labs(
    title = "The bootstrap distribution and all four CIs",
    x      = "Difference in means, East minus West (percentage points)",
    y      = "Density"
  ) +
  theme_intror()

```

The bootstrap distribution and all four CIs



Concluding remarks

You’ve now built confidence intervals three different ways — analytically, by simulation, and by bootstrap — for both a single mean and a difference in means, and seen R’s own shortcut (`t.test()`) reproduce the analytical result. There’s always a trade-off between how *confident* we want to be and how *precise* the resulting statement is: a wider interval is more likely to capture the truth, but says less. The real fix for that tension is a bigger sample, not a wider interval. Week 6 picks this thread back up and turns “does zero sit outside this interval?” into a formal hypothesis test, with its own machinery for deciding how surprised to be.

Try it: a 99% CI for the AfD’s polled mean share

Your task: using the same final-stretch polling data and the analytical method, build a 99% (not 95%) confidence interval for the AfD’s mean polled share. How does it compare to the 95% interval above?

```
ci_analytical_99 <- mean_share + qnorm(c(0.005, 0.995)) * se_share
ci_analytical_99
```

> OUTPUT

```
[1] 20.34027 21.09973
```

Try it: bootstrap CI for a share of strong polls

Your task: build a bootstrap 95% CI for the *proportion* of these same 25 polls that put the AfD at 20.5% or higher, reusing the same resampling pattern as the mean's bootstrap CI above.

```
afd_strong <- afd_shares >= 20.5

boot_prop <- expand_grid(trial = 1:n_boot, row = 1:n_polls) |>
  group_by(trial) |>
  summarize(boot_p = mean(sample(afd_strong, n(), replace = TRUE)), .groups = "drop")

boot_prop_values <- boot_prop |> pull(boot_p)
ci_prop_bootstrap <- quantile(boot_prop_values, c(0.025, 0.975))
ci_prop_bootstrap
```

> OUTPUT

```
2.5% 97.5%
0.36  0.76
```

Try it: repeat the difference-in-means for the Greens

Your task: redo the East/West difference-in-means comparison using Green vote share instead of AfD's (`t.test()` is enough). Does the gap point the same direction?

```

gerda_regions_gruene <- gerda_2025 |>
  filter(state %in% c(west_codes, east_codes)) |>
  mutate(
    region = case_when(
      state %in% west_codes ~ "West",
      state %in% east_codes ~ "East"
    ),
    gruene_pct = gruene * 100
  ) |>
  select(county_code, state, region, gruene_pct)

gruene_east <- gerda_regions_gruene |>
  filter(region == "East") |>
  pull(gruene_pct)
gruene_west <- gerda_regions_gruene |>
  filter(region == "West") |>
  pull(gruene_pct)

t.test(gruene_east, gruene_west)

```

> OUTPUT

Welch Two Sample t-test

```

data: gruene_east and gruene_west
t = -14.743, df = 150.57, p-value < 0.00000000000000022
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -7.184290 -5.486247
sample estimates:
mean of x mean of y
 4.890062 11.225330

```